

БЫКИ И КОРОВЫ

Групповой проект

«Игра Быки и Коровы»

Выполнен Чупрасовой Алёной & Коваленко Александрой

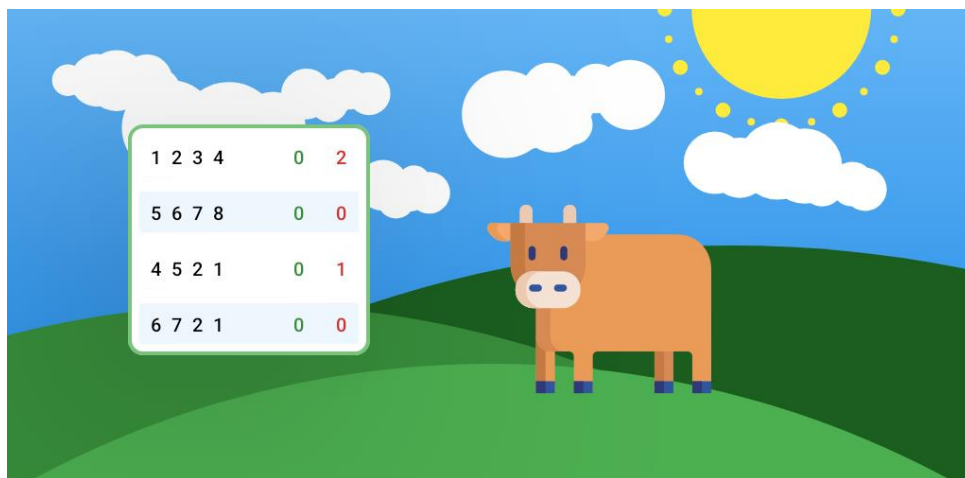
ВВЕДЕНИЕ

Ваша задача в рамках данного проекта — реализовать классическую логическую игру «Быки и Коровы» в среде моделирования Logisim. Система должна базироваться на полном ядре процессора CdM-8 с Гарвардской архитектурой, подразумевающей раздельное хранение кода программы и данных. Взаимодействие между процессором и периферией организуется через шину ввода-вывода с отображением на память (Memory-Mapped I/O).

В отличие от классических игр с полем, интерфейс данной системы строится на принципе цифрового ввода. Игровая панель (Game Pad) представляет собой набор клавиш для ввода цифр и группу шестнадцатеричных индикаторов для отображения текущего состояния игры: введенного числа, результатов проверки (количества «быков» и «коров») и счетчика попыток. Процессор CdM-8 выступает в роли ведущего: принимает ходы игрока, анализирует их и выдает результат.

Ограниченность ресурсов команд и данных требует от разработчиков тщательной оптимизации алгоритмов. Рекомендуется сосредоточиться на создании стабильно работающей базовой версии игры, где корректно реализованы правила подсчета совпадений, прежде чем переходить к сложным расширениям.

Быки и коровы — логическая игра, в ходе которой за несколько попыток игрок должен определить, задуманное число. В своё время эта игра была очень популярна среди студентов и школьников. Игра развивает умение анализировать и логически мыслить.



*один из примеров внешнего вида

ПРАВИЛА ИГРЫ

Задуманы четыре различные цифры (от 0 до 9). У вас есть 25 попыток, чтобы отгадать все цифры и их расположение. Если же игрок не справился с этим за заданное количество попыток – игра считается проигранной.

Попытка — это 4-значное число с неповторяющимися цифрами. В ответ вам будет показано число отгаданных цифр, стоящих на своих местах (число быков) и число отгаданных цифр, стоящих не на своих местах (число коров).

Пример: Задумано тайное число: «3219». Попытка: «2310». Результат: один «бык» (одна цифра «1» угадана вплоть до позиции) и две «коровы» (две цифры: «2» и «3» — угаданы, но на неверных позициях). В нашей вариации вы вводите число с помощью кнопок и нажимаете «ОК» для проверки. Для следующего хода нужно нажать «Nv» (новый ввод) для сброса предыдущего хода. Но помните, чем меньше попыток вы используете, тем лучше!

АППАРАТНАЯ РЕАЛИЗАЦИЯ

Аппаратная часть системы проектируется как совокупность функциональных блоков, подключенных к шинам процессора. Центральным элементом является процессор CdM-8, к которому подключаются блок генерации кода, модуль ввода и модуль вывода. Используется Гарвардская архитектура.

Блок генерации кода (Gen_Code) Данный модуль отвечает за формирование стартового состояния игры. При активации сигнала сброса или начала новой игры, блок запускает генератор псевдослучайных чисел. Важной особенностью является логика контроля уникальности: генератор должен обеспечить формирование четырехзначного числа, в котором ни одна цифра не повторяется. Сформированный код передается в регистры памяти данных для последующего сравнения.

Модуль ввода (Input Unit) Интерфейс ввода предназначен для приема сигналов от кнопочной панели (цифры 0-9, кнопка подтверждения и кнопка нового ввода). Введенные цифры последовательно записываются в буферный регистр.

Модуль вывода (Output Unit) Данный блок отвечает за визуализацию игрового процесса. Он включает в себя шестнадцатеричные индикаторы.

Индикаторы отображают: количество «быков» и «коров» после проверки хода, а также номер текущей попытки и флаг победы. Также предусмотрены матрицы, сигнализирующие о победе (загорание матрицы с надписью «Win!») или о проигрыше (надпись «Loser»).

Модуль bcd Коррекционный блок. Во избежание ошибок при подсчёте попыток.

Модуль block Схема защиты от повторного ввода. Поскольку правила игры запрещают использование одинаковых цифр в одном ходе, аппаратная или программная логика должна блокировать сигнал от кнопки, если соответствующая цифра уже была введена в текущем четырехзначном числе.

ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

Программная часть пишется на языке ассемблера для процессора CdM-8 и управляет всем циклом игры. Программа должна быть структурирована таким образом, чтобы эффективно использовать ограниченную память команд.

ОСНОВНЫЕ ИНСТРУКЦИИ ПРОЦЕССОРА CdM-8

Ниже приведен список команд и директив ассемблера, которые стоит использовать для реализации логики игры «Быки и Коровы». Команды сгруппированы по назначению.

Директивы ассемблера

asect <addr> Устанавливает адрес, с которого будет размещаться следующий блок кода или данных. Используется для разделения кода программы, переменных игры и портов ввода-вывода.

ds <n> Резервирует *n* байт памяти для хранения переменных. Значения ячеек инициализируются нулями.

Работа с данными и регистрами

ldi rX, val Загружает в регистр rX непосредственное число или адрес переменной. Это самая частая команда для подготовки к чтению или записи.

ld rA, rB Загружает значение из памяти. Процессор берет адрес из регистра rA, читает значение по этому адресу и помещает его в регистр rB.

st rA, rB Сохраняет значение в память. Процессор берет значение из регистра rB и записывает его в ячейку памяти, адрес которой находится в регистре rA.

Арифметические операции

inc rX Увеличивает значение в регистре rX на единицу. Используется для подсчета совпадений.

Управление потоком и условия

cmp rA, rB Сравнивает значения двух регистров. Результат не сохраняется, но обновляются флаги процессора для последующей проверки условий.

tst rX Тестирует значение регистра на ноль (либо же от выбранного флага). Используется для проверки флагов окончания игры.

br <label> Безусловный переход. Выполняет прыжок к указанной метке. Используется для организации бесконечных циклов игры.

if ... is eq ... fi Структурная конструкция условного перехода. Выполняет блок кода внутри if, только если предыдущее сравнение (cmp) показало равенство (Equal).

halt Останавливает выполнение программы. Процессор прекращает работу.

Основной алгоритм работы программы начинается с инициализации системы. Программа входит в главный игровой цикл. На каждом витке цикла процессор ожидает ввода числа от игрока. Как только ввод завершен (нажата кнопка подтверждения), запускается подпрограмма анализа.

Подпрограмма сравнения должна последовательно проверить введенные цифры. Сначала проводится поиск полных совпадений (быков) — сравнение цифры ввода с цифрой кода на той же позиции. Найденные совпадения помечаются как обработанные. Затем проводится поиск частичных совпадений (коров) — поиск цифры ввода среди необработанных цифр кода. Результаты подсчета записываются в соответствующие ячейки памяти. Также реализуется прямое чтение по адресам.

После подсчета программа обновляет состояние дисплеев, выводя результаты на индикаторы. Если количество быков равно четырем, устанавливается флаг победы, активируется сигнал завершения игры, и программа переходит в состояние ожидания сброса. Если победа не достигнута, счетчик попыток увеличивается, и система возвращается к ожиданию нового ввода.

РЕКОМЕНДУЕМЫЙ ПОРЯДОК РАЗРАБОТКИ

Для успешной реализации проекта рекомендуется придерживаться следующей последовательности действий. Начните с создания и тестирования отдельных аппаратных блоков в Logisim. Сначала реализуйте блок ввода с механизмом блокировки повторных цифр, убедившись, что каждая цифра фиксируется только один раз за ход. Затем соберите ядро, модуль вывода и вспомогательные подсхемы и проверьте корректность отображения данных на шестнадцатеричных индикаторах.

Параллельно с аппаратной частью начните разработку программного кода. Напишите подпрограммы для работы с памятью и простые алгоритмы сравнения. Убедитесь, что логика подсчета «быков» и «коров» работает верно на тестовых данных.

На завершающем этапе интегрируйте программу в собранную схему Logisim. Проведите комплексное тестирование: проверьте генерацию кода, процесс ввода, корректность подсчета результатов и реакцию системы на победу. При необходимости оптимизируйте код ассемблера для экономии памяти.

РЕКОМЕНДАЦИИ ПО ОТЛАДКЕ

Этап 1: Тестирование отдельных модулей

Начните отладку с проверки работы генератора случайного кода. Убедитесь, что при каждом запуске генерируется новое число, состоящее из четырех различных цифр. Для этого можно временно вывести сгенерированный код на индикаторы и проверить несколько последовательных запусков.

Протестируйте блок ввода отдельно от остальной системы. Последовательно нажимайте кнопки цифр и убедитесь, что:

- каждая цифра корректно фиксируется;
- повторное нажатие той же цифры игнорируется;
- после ввода четырех цифр при нажатии кнопки «ОК» формируется сигнал готовности.
- для следующего ввода нажимается кнопка «Nv».

Этап 2: Проверка подпрограмм сравнения

Тестирование алгоритма подсчета «быков» и «коров». Создайте тестовые наборы данных с известными результатами:

- Код 1234, ввод 1234 - ожидаемый результат: 4 быка, 0 коров
- Код 1234, ввод 4321 - ожидаемый результат: 0 быков, 4 коровы
- Код 1234, ввод 1456 - ожидаемый результат: 1 бык, 1 корова
- Код 1234, ввод 5678 - ожидаемый результат: 0 быков, 0 коров

Этап 3: Интеграционное тестирование

После успешной проверки отдельных компонентов подключите программу к аппаратной схеме в Logisim. Проведите полную игровую сессию:

1. Убедитесь, что при старте генерируется код;
2. Введите тестовое число и проверьте корректность отображения;
3. Проверьте правильность подсчета результатов;
4. Убедитесь, что индикаторы победы/поражения срабатывают в нужный момент.

Типичные проблемы и способы их решения:

Если индикаторы не отображают данные, проверьте правильность подключения шин адреса и данных, а также корректность сигналов управления чтением/записью.

Если подсчет «быков» и «коров» дает неверный результат, убедитесь, что алгоритм правильно помечает обработанные цифры и не учитывает их повторно.

Если система зависает, проверьте наличие безусловных переходов в основном цикле и корректность условий выхода из циклов ожидания.